

Module 14: Streams

A stream is a pipeline, not a collection. It holds no data and it is consumed once. Nearly everything surprising about streams follows from those two facts.

Module 13 built the parts. Every lambda in a stream pipeline is one of the interfaces from that module, so this is where they start paying off.

What You'll Learn

- Pipeline anatomy, and laziness shown rather than asserted
- Why elements flow through one at a time, not stage by stage
- Collectors, with `groupingBy` as the one that justifies the API
- Gatherers, which fill the gap where custom intermediate operations belong
- Parallel streams, when they help, and the trap that makes them dangerous

Coming From Another Language

	Python	Java
Transform	comprehension, <code>map</code>	<code>.map(...)</code>
Filter	comprehension <code>if</code> , <code>filter</code>	<code>.filter(...)</code>
Lazy	generators	streams, always
Reuse	a list can be iterated twice	a stream cannot
Group	<code>itertools.groupby</code> needs sorting	<code>Collectors.groupingBy</code> does not
Parallel	<code>multiprocessing</code>	<code>.parallel()</code> , and think first

The reuse row is the one that bites. A Python list comprehension gives you a list. A Java stream gives you a pipeline that evaporates once used.

The Lesson

Anatomy

```
words.stream()           // SOURCE
    .filter(w -> w.length() > 4) // INTERMEDIATE, lazy
    .map(String::toUpperCase) // INTERMEDIATE, lazy
    .toList();            // TERMINAL, runs everything
```

Intermediate operations return a stream and do nothing. The terminal operation is what actually runs the pipeline.

Laziness, demonstrated

From `StreamBasics.java`, building a pipeline with a `peek` in it prints nothing:

```
pipeline built. notice nothing was peeked.  
now adding a terminal operation:  
peeked at banana  
...
```

Laziness is what makes **short-circuiting** possible:

```
examined 1  
examined 2  
examined 3  
found Optional[3]
```

Elements 4 and 5 were never touched. It is also why an infinite stream works:

```
Stream.iterate(1, n -> n + 1).map(n -> n * n).limit(5).toList()
```

Element-wise, not stage-wise

A common misreading is that `filter` runs over everything, then `map` runs over everything. It does not. Each element goes through the **whole pipeline** before the next one starts:

```
filter sees a  
map sees a  
forEach sees a  
filter sees b  
...
```

Collectors

From `CollectingResults.java`, `groupingBy` alone justifies learning the API:

```
STAFF.stream().collect(Collectors.groupingBy(Employee::department))
```

The **downstream collector** is the second argument, and it is where the API stops being verbose and starts being worth it:

```
groupingBy(Employee::department, Collectors.counting())  
groupingBy(Employee::department, Collectors.averagingInt(Employee::salary))  
groupingBy(Employee::department, TreeMap::new, Collectors.counting()) // ordered
```

Without the map factory you get a `HashMap`, whose order is undefined. Module 12 applies here too.

`partitioningBy` splits into exactly two groups and always has both keys, even when one side is empty.

`toMap` **throws on a duplicate key** rather than overwriting:

```
duplicate key threw: Duplicate key engineering ...
```

That is usually what you want. Supply a merge function to say otherwise.

`.toList()` **on the stream returns an unmodifiable list** and is the modern form.

`Collectors.toList()` returns a modifiable list of unspecified type. Prefer the former.

Gatherers

Finalised in JDK 24, so no flags here. They fill a real gap: you could always write a custom **terminal** operation as a `Collector`, but never a custom **intermediate** one. Anything stateful across elements meant leaving the stream.

From `UsingGatherers.java`:

Gatherer	Result on <code>1,2,3,4,5</code>
<code>windowFixed(2)</code>	<code>[[1, 2], [3, 4], [5]]</code>
<code>windowSliding(2)</code>	<code>[[1, 2], [2, 3], [3, 4]]</code>
<code>scan(() -> 0, Integer::sum)</code>	<code>[1, 3, 6, 10]</code>

`windowFixed` keeps the short final window rather than dropping it, which is what you want for batching so no records go missing. `scan` is `reduce` that emits every intermediate value, which is what a running balance needs.

`mapConcurrent(n, fn)` maps with a bounded concurrency limit **and preserves order**, running each on a virtual thread. That is a better default than `.parallel()` for IO-shaped work, for reasons the next section explains.

When to reach for one: state across elements, or emitting a different number of elements than you consumed. One in, one out, no memory is just `map`.

Parallel streams

From `ParallelStreams.java`. Adding `.parallel()` is one word, and it is almost never the right call.

The trap, run three times on the same machine:

```
the list now holds: 23839
the list now holds: 22694
the list now holds: 84097
```

100,000 elements were added to an `ArrayList` in parallel. Every run lost a different number, and a fourth run might throw instead, when one thread resizes the backing array while another is mid-write.

It is not reliably wrong, which is worse than being reliably wrong. On a small input or an unloaded machine it often works. That is how this reaches production.

The fix is not a lock, it is `collect` :

```
IntStream.range(0, 100_000).parallel().boxed().toList(); // 100000, always
```

Each thread accumulates into its own container and the results merge at the end, so nothing is shared while being written. Synchronising `add` would also be correct and would be *slower* than the sequential version, because every thread would queue on one lock.

`forEach` does not preserve order in parallel; `forEachOrdered` does.

Consider <code>.parallel()</code>	Avoid it
large N, genuinely expensive per element	small N: fork/join overhead dominates
source splits cheaply: arrays, <code>ArrayList</code> , <code>IntStream.range</code>	<code>LinkedList</code> or iterator sources
no shared state, no ordering requirement	anything doing IO

The IO case deserves emphasis. The common pool is sized for CPU cores, and blocking it starves every other parallel stream in the JVM. Use `Gatherers.mapConcurrent` or Module 19's virtual threads instead.

The honest default: do not write `.parallel()` until a profiler names this pipeline as the bottleneck.

Single use

`StreamsAreSingleUse.java` :

```
IllegalStateException: stream has already been operated upon or closed
```

A stream holds no elements of its own to replay. Keep the **source**, not the stream, and a collection will give you as many streams as you want.

Run It

```
java modules/14-streams/examples/StreamBasics.java
java modules/14-streams/examples/CollectingResults.java
java modules/14-streams/examples/UsingGatherers.java
java modules/14-streams/examples/ParallelStreams.java

# Fails on purpose.
java modules/14-streams/examples/StreamsAreSingleUse.java

./scripts/verify-examples.sh modules/14-streams
```

Common Mistakes

Storing a stream in a field or variable and using it twice. Store the source.

Mutating a shared collection from `forEach`. Use `collect`. In parallel it is a data race; even sequentially it is harder to read than the collected version.

Reaching for `.parallel()` to make something faster. Measure first. It is frequently slower and occasionally wrong.

Blocking inside a parallel stream. You have just taken threads from a pool the whole JVM shares.

Using `peek` for real side effects. It is a debugging window. Implementations are allowed to skip it when the result is not needed.

Forgetting `toMap` throws on duplicates. Supply a merge function when keys can repeat.

A pipeline longer than the loop it replaced. Streams are for expressing *what*, not a rule that loops are forbidden. If the loop is clearer, write it.

Key Takeaways

- **A stream is a lazy pipeline, consumed once.** Keep the source, not the stream.
- **Nothing runs until the terminal operation**, which is what enables short-circuiting and infinite sources.
- **Elements flow one at a time through the whole pipeline**, not stage by stage.
- **`groupingBy` with a downstream collector** is the most useful thing in the API.
- **Gatherers give you custom intermediate operations**, for windowing and running totals.
- **`.parallel()` with shared mutable state loses data unpredictably.** Use `collect`, and measure before parallelising at all.

Homework

[homework/README.md](#)

Analyse a log with a stream pipeline, use a gatherer, and reproduce the parallel data race. Reference solution in [solutions/14-streams/](#).